Novapolis Résilience

Rapport de projet d'algorithmique des graphes · ING-APP1-BDML

Jakub Potaczala (partie A), Royston Gnanapragasam (partie B), Ravichandran Arumugadas (partie C).

Novembre 2026

Résumé

Novapolis veut savoir si ses secours et son réseau de fibre résistent quand une route ferme ou qu'un câble casse. Nous avons construit un outil Python, sans bibliothèque de graphes, qui charge des données CSV validées, modélise la ville par deux graphes, applique des scénarios de perturbation sur une copie, et mesure leurs effets avec des indicateurs vérifiables.

Les neuf algorithmes du sujet sont implémentés : DFS, BFS, Dijkstra, Bellman–Ford, Floyd–Warshall, Kruskal, Prim, Union-Find et Kosaraju. Les résultats principaux sont les suivants.

Accessibilité : dans le jeu réaliste, les 120 sites sont atteignables depuis les 4 casernes, et le réseau est fortement connexe. Dans le jeu généré de 1000 sites, le site `S1000` est atteignable mais on ne peut pas en repartir.

Itinéraires : Dijkstra, Bellman–Ford et Floyd–Warshall donnent exactement les mêmes distances. Fermer `C1 → Q1` dans le petit graphe fait passer un trajet de 4 à 10 min (+150 %), comme le calcul fait à la main.

Points faibles : un test de résistance, qui coupe chacune des 53 routes utilisées, montre que la route la plus critique du jeu réaliste est `CAS_04 ↔ QUA_084` (117,5 min de retard cumulé). Le scénario écrit à la main, le « pont central », ne retarde aucune demande.

Fibre : l'arbre couvrant minimal du jeu réaliste coûte 2222,8 k€. Deux tronçons n'ont aucune solution de secours en cas de panne.

Performances : Kruskal, Prim et Dijkstra suivent leur complexité théorique. Bellman–Ford s'arrête bien avant son pire cas. L'écart du BFS sur les gros graphes s'explique par la disposition des données en mémoire.

L'outil se pilote en ligne de commande et par un site local interactif. 377 tests automatiques vérifient le tout, et chaque résultat de ce rapport se régénère avec les commandes du README. Toutes les données sont synthétiques.

1. Contexte et modélisation

La ville pose trois questions : quels quartiers sont mal accessibles depuis une caserne ou un hôpital, quels itinéraires recommander quand des routes sont lentes ou fermées, et comment déployer la fibre au moindre coût en gardant une solution de secours.

Les deux réseaux partagent les mêmes sites mais n'obéissent pas aux mêmes règles. Nous les modélisons donc par deux graphes distincts.

	Réseau routier	**Réseau de fibre**
Graphe	Orienté, pondéré : un arc = un trajet autorisé dans un sens	Non orienté, pondéré : une arête = un tronçon réalisable
Poids	Durée en minutes (`temps_normal` ou `temps_pointe`)	Coût de construction en k€

	Réseau routier	Réseau de fibre
Pourquoi	Les sens uniques rendent la relation asymétrique	Un câble transporte les données dans les deux sens
Question	Plus court chemin, accessibilité	Arbre couvrant de poids minimal

Poids positifs. Dans tous les scénarios métier, une durée est positive. Des coûts négatifs n'apparaissent que dans deux jeux de test artificiels, destinés à vérifier Bellman–Ford et la détection des circuits absorbants (section 5). Ils ne représentent jamais une durée réelle.

Statut des tronçons. Chaque arc porte un statut. Une route `ferme` est ignorée par tous les algorithmes ; une route en `travaux` reste praticable mais est signalée dans l'itinéraire. Pour la fibre, un tronçon `ferme` ou en `travaux` n'est jamais utilisé pour une réparation. Cette règle est la même dans tous les modules.

Trois jeux de données, tous synthétiques, répondent aux exigences du sujet.

Jeu	Sites	Arcs routiers	Tronçons de fibre	Demandes	Rôle
`data/petit`	8	19	11	4	Vérification à la main
`data/realiste`	120	431	163	25	Ville documentée
`data/genere`	1000	3000	2433	50	Expériences, graine 42

2. Données, validation et architecture

Formats. Les données sont cinq fichiers CSV décrits champ par champ dans le README : `sites.csv`, `routes.csv`, `fibre.csv`, `demandes.csv` et `scenarios.csv`. Un scénario combine quatre actions : `fermer_route`, `modifier_temps`, `heure_pointe` et `fermer_fibre`.

Validation. Avant tout calcul, `io_donnees.py` rejette les données incorrectes avec un message qui donne la ligne en cause, par exemple « routes.csv ligne 14 : site S99 inconnu ». Il détecte 18 types d'erreurs, dont les identifiants en double, les arcs vers un site inexistant, les colonnes manquantes, les valeurs non numériques et les temps négatifs dans un jeu métier. Les jeux de test à coûts négatifs ne se chargent qu'avec une autorisation explicite.

Scénarios sur copie. `appliquer_scenario` travaille toujours sur `graphe.copie()`, une copie profonde. Le graphe de référence ne change jamais, et un test le vérifie après chaque application. C'est ce qui permet de mesurer un retard par rapport à la situation normale. Chaque action ne vise que son réseau : une panne de fibre ne ferme pas la route du même couple de sites.

Structures de données. Les graphes sont stockés en listes d'adjacence : un réseau urbain est creux (environ 3 arcs par sommet), la mémoire est donc en O(|V| + |E|) et les voisins d'un sommet s'obtiennent en O(degré), ce que demandent BFS, DFS, Dijkstra et Prim. Une matrice n × n n'est construite que dans Floyd–Warshall. S'y ajoutent un tas binaire (`heapq`), une file (`deque`), une pile explicite et un Union-Find. Aucun algorithme n'est récursif, car Python s'arrête vers 1000 appels imbriqués.

Contrat d'interface. Dès le départ, l'équipe a fixé un format commun : tous les algorithmes de plus court chemin renvoient `(dist, pred)`, avec `inf` pour un sommet inaccessible et `-inf` pour un sommet dont la distance peut diminuer sans fin. Une seule fonction, `reconstruire_chemin`, sert donc aux trois algorithmes.

3. Module 1 – Diagnostic d'accessibilité

Le module 1 répond à la première question de la ville : les secours peuvent-ils atteindre tous les lieux ? Il est implémenté dans `src/parcours.py`.

3.1 Trois notions à ne pas confondre

Le réseau routier étant orienté, « tout est relié » peut vouloir dire trois choses différentes. Le sujet demande de préciser la notion utilisée pour chaque indicateur.

Notion	Définition	Algorithme	Indicateur produit
Accessibilité depuis une caserne	v est accessible s'il existe un chemin orienté d'une caserne vers v	BFS multi-sources	Sites atteignables / non atteignables, caserne d'origine, part des sites sensibles atteints
Connexité (simple)	Le graphe est connexe si, en **oubliant le sens** des arcs, tous les sommets sont reliés	BFS sur le graphe symétrisé	Nombre de composantes connexes
Connexité forte	u et v sont dans la même composante si u → v **et** v → u	Kosaraju (2 DFS)	Nombre de CFC, sites hors de la grande CFC

Les implications ne vont que dans un sens : fortement connexe implique accessible depuis n'importe quelle caserne, qui implique connexe. Pour les secours, l'indicateur pertinent est l'accessibilité : un véhicule doit pouvoir **atteindre** le site. La connexité forte répond à une question plus exigeante : le véhicule peut-il aussi **revenir** ?

3.2 Algorithmes

Tous les parcours sont itératifs et en O(|V| + |E|) : `dfs` (version pile du cours), `bfs` et `chemin_min_arcs` (chemin qui minimise le **nombre d'arcs**, pas la durée), `accessibilite_multi_sources` (toutes les casernes dans la file au départ, chaque site hérite de sa caserne d'origine), `trouver_circuit` (DFS à trois couleurs : un arc vers un sommet gris ferme un circuit), `sites_isoles` et `sites_non_atteignables`, et `composantes_fortement_connexes` (Kosaraju : deux DFS, le second sur le graphe inversé).

3.3 Résultats

Jeu	Atteignables depuis les casernes	Sites sensibles atteints	Composantes connexes	Composantes fortement connexes
Petit	8 / 8	2 / 2	1	1
Réaliste	120 / 120	15 / 15	1	1
Généré	1000 / 1000	tous	1	**2**

Sur le petit jeu, l'arc `Q2 → Q3` est à sens unique, mais le graphe reste fortement connexe grâce au retour `Q3 → H1 → Q1 → Q2`. Les distances en nombre d'arcs obtenues par BFS correspondent aux valeurs calculées à la main.

Le jeu généré montre pourquoi distinguer les notions : tous les sites sont accessibles, mais le graphe n'est **pas** fortement connexe. Le site `S1000` n'a aucune route sortante : un véhicule peut y aller, mais pas en repartir. Seule la connexité forte détecte ce cas.

4. Module 2 – Itinéraires des secours

Pour chaque demande d'intervention, le module 2 choisit la caserne la plus proche et calcule le trajet le plus rapide. Il est implémenté dans `src/chemins.py`.

4.1 Trois algorithmes, un même format

Algorithme	Mise en œuvre	Complexité	Usage
Dijkstra	Tas binaire `heapq` et suppression paresseuse : une entrée (d, u) avec d supérieur à `dist[u]` est périmée et ignorée. Refuse tout poids négatif en nommant l'arc fautif	O((|V| + |E|) log |V|)	Cas métier, durées positives
Bellman–Ford	|V| − 1 passes de relâchement, arrêt anticipé si une passe ne change rien, puis une passe de détection	O(|V| · |E|)	Poids négatifs possibles (module 3)
Floyd–Warshall	Matrice L des distances et matrice P des prédécesseurs. Un circuit absorbant se lit sur la diagonale : L[i][i] < 0	O(|V|³) en temps, O(|V|²) en mémoire	Toutes les paires, petits graphes

Pourquoi Dijkstra refuse les poids négatifs. Il fixe définitivement un sommet dès qu'il sort du tas. Un arc négatif découvert plus tard pourrait faire baisser sa distance, et le résultat serait faux sans aucun message. Lever une erreur est plus sûr que renvoyer un résultat faux.

Choix de la caserne. Au lieu de lancer un Dijkstra par caserne, toutes les casernes entrent dans le tas à distance 0 (Dijkstra multi-sources). Chaque sommet hérite de la caserne d'origine de son prédécesseur : on obtient pour chaque site la caserne la plus proche et le chemin, pour le coût d'un seul Dijkstra.

4.2 Vérification

Sur le petit jeu, les trois algorithmes retrouvent les distances calculées à la main. `comparer_algorithmes` vérifie automatiquement qu'ils donnent les mêmes distances : toutes les paires du petit jeu, les casernes du jeu réaliste (480 distances, aucun écart) et trois sources du jeu généré, sans Floyd–Warshall, trop lent à 1000 sommets.

4.3 Effet des fermetures

Un scénario est appliqué sur une copie du graphe, puis `comparer_scenario` donne pour chaque demande le coût avant et après, le retard absolu, le retard relatif (après − avant) / avant, l'accessibilité et le nouveau chemin.

Petit jeu, fermeture de `C1 → Q1`. La demande D1 passe de 4 à 10 min par `C1 → E1 → Q2 → Q1` (5 + 2 + 3), soit +6 min et +150 %, comme le calcul fait à la main. D4 passe de 7 à 13 min par un détour ; D2 et D3 ne changent pas, et aucune demande ne devient inaccessible.

Synthèse des scénarios routiers (retard moyen calculé sur toutes les demandes accessibles, retards nuls compris) :

Jeu	Scénario	Demandes retardées	Retard moyen	Retard maximal
Petit	Fermeture de C1 → Q1	2 / 4	3,0 min	6,0 min
Petit	Heure de pointe	4 / 4	4,25 min	7,0 min
Réaliste	« Pont central » (CAS_01 ↔ HOP_01)	0 / 25	0	0
Réaliste	Fermeture de CAS_01 ↔ QUA_046	5 / 25	2,9 min	29,0 min
Réaliste	Heure de pointe	25 / 25	21,3 min	40,6 min

La fermeture du « pont central » n'a **aucun effet** : aucun plus court chemin de demande ne l'emprunte. Une route n'est importante que si les itinéraires l'utilisent vraiment, ce qui motive le test de résistance de la section 7. L'heure de pointe, elle, ralentit toutes les demandes sans changer les casernes affectées.

La visualisation de la partie B (`src/visu.py`, pygame) montre ce cas, route fermée en pointillé rouge et nouveau chemin en vert ; elle est reprise dans le site (section 9).

5. Module 3 – Circuits absorbants

Un circuit absorbant est une boucle dont le coût total est négatif : chaque tour fait baisser la distance, si bien qu'aucun plus court chemin n'existe. Les durées métier étant positives, Novapolis n'en contient aucun. Deux jeux de test **artificiels** servent donc à vérifier que Bellman–Ford et Floyd–Warshall les détectent. Leurs coûts négatifs ne représentent aucune durée réelle, ce que rappellent leur README et la sortie de la commande.

5.1 Les deux graphes de test

`data/absorbants/accessible` : circuit S2 → S3 → S4 → S2 de coût 2 + 3 − 6 = −1, atteignable depuis S1. Pièges : S5 n'est pas sur le circuit mais vient après lui (−∞) ; S6 et S7 gardent des distances finies (4 et 6) ; S8 n'est jamais atteint (+∞).

`data/absorbants/non_accessible` : circuit S4 → S5 → S6 → S4 de coût 1 + 1 − 4 = −2, hors de portée de S1. Piège : depuis S1, Bellman–Ford donne des distances normales (S3 à 5), alors que le circuit existe.

Les réponses attendues sont écrites dans un fichier `resultats_attendus.json` par jeu. Elles sont vérifiées **sans Bellman–Ford**, par force brute : les tests énumèrent tous les circuits et tous les chemins simples. Les réponses ne dépendent donc pas du code qu'elles servent à tester.

5.2 Détection

La détection (`src/absorbants.py`) s'appuie sur les fonctions de la partie B et combine trois méthodes.

1. **Depuis une source**, `bellman_ford_detail` fait |V| − 1 passes, puis une passe supplémentaire : un sommet encore relâchable est influencé par un circuit accessible.
2. **Pour un circuit hors de portée**, Bellman–Ford depuis la source ne voit rien, car les sommets du circuit restent à +∞. On ajoute alors une super-source virtuelle, reliée à tous les sommets avec un coût 0 : tout devient atteignable, et le circuit est détecté.
3. **Pour nommer les sommets du circuit**, on lit la diagonale de Floyd–Warshall : L[i][i] < 0 si i est sur un circuit absorbant. Cette méthode est limitée à 400 sommets, en raison de son coût en $O(n^3)$.

5.3 Distances − ∞

Tout sommet atteignable depuis un sommet encore relâchable à la passe |V| reçoit −∞, par propagation en BFS. Une distance qui peut diminuer sans fin n'est **jamais** affichée comme un nombre fini.

Les quatre cas donnent exactement les réponses attendues, pièges compris. Depuis S1, la distance de S3 reste bien 5 : une super-source verrait S3 diminuer sans fin, mais cette valeur ne doit pas être reportée sur la distance depuis S1. Des tests sur 40 graphes aléatoires vérifient aussi que la super-source et la diagonale de Floyd–Warshall trouvent un circuit exactement quand la force brute en trouve un.

6. Module 4 – Conception du réseau de fibre

Le module 4 répond à la troisième question : relier tous les sites en fibre au moindre coût, puis réagir à une panne. Il est implémenté dans `src/arbres.py`.

6.1 Union-Find, Kruskal et Prim

Union-Find représente chaque composante par un arbre identifié par sa racine. Avec la **compression de chemin** et l'**union par rang**, chaque opération est quasi constante, en O(α(n)) amorti. La recherche de racine est itérative (testée sur une chaîne de 100 000 éléments).

Kruskal trie les arêtes par (coût, u, v) et garde chacune si ses deux extrémités ne sont pas déjà reliées. Sinon, elle fermerait une boucle et elle est rejetée. Si le graphe n'est pas connexe, on obtient naturellement une forêt couvrante. Complexité : O(|E| log |E|), dominée par le tri.

Prim fait grandir un arbre à partir d'un sommet, en prenant toujours l'arête la moins chère qui en sort, grâce à un tas binaire avec suppression paresseuse. Il est relancé depuis chaque sommet non visité pour obtenir une forêt. Complexité : O(|E| log |V|).

Jeu	Tronçons possibles	Tronçons gardés	Coût Kruskal	Coût Prim	Même arbre ?
Petit	11	7	35,0 k€	35,0 k€	Oui
Réaliste	163	119	2222,8 k€	2222,8 k€	Oui
Généré	2433	999	9079,5 k€	9079,5 k€	Non, 1 tronçon diffère

Sur le petit jeu, le coût de 35 et les 7 tronçons correspondent au calcul à la main. Sur 30 graphes aléatoires, le coût de Kruskal est aussi comparé à celui de tous les arbres couvrants possibles (force brute).

6.2 Égalités de coûts

Dans le jeu généré, Kruskal garde `S0037 – S0803` et Prim garde `S0390 – S0474`. Les deux coûtent 15,5 k€ : deux arbres différents ont donc exactement le même coût minimal. Le jeu `data/egalites` illustre ce cas à la main : un cycle A – C – E – B – D – A de 5 tronçons, dont deux à 2 k€. Retirer l'un ou l'autre donne un arbre de 5 k€. Un test qui énumère tous les arbres en trouve exactement deux de coût minimal.

L'arbre couvrant minimal est unique **si tous les coûts sont distincts** (vérifié sur 50 graphes aléatoires). La réciproque est fausse : le petit jeu contient des coûts égaux, et son arbre minimal est pourtant unique. En cas d'égalité, l'algorithme ne dit pas quel tronçon est le meilleur : il choisit selon l'ordre de tri. La ville pourrait départager avec d'autres critères, comme le risque d'inondation.

6.3 Panne d'un tronçon et raccordement

Quand un tronçon de l'arbre casse, l'arbre se coupe en deux. Chaque côté est étiqueté par BFS, puis on cherche **dans le graphe initial**, sans le tronçon en panne ni les tronçons indisponibles, l'arête la moins chère qui relie les deux côtés. Par la propriété de coupure, cette arête redonne un arbre de coût minimal. Un test le confirme : sur 30 graphes aléatoires, pour chaque tronçon de l'arbre, le coût obtenu est égal à celui d'un Kruskal relancé sans ce tronçon. Si aucune arête ne relie les deux côtés, la panne est signalée comme irréparable.

Jeu	Tronçon en panne	Résultat
Petit	Q2 – Q3 (8 k€)	Raccordement par C2 – E1 pour 9 k€, l'arbre passe de 35 à 36 k€
Généré	S0001 – S0776 (7,7 k€)	Raccordement par S0001 – S0346 pour 11,2 k€

Jeu	Tronçon en panne	Résultat
Réaliste	ECO_05 – QUA_074	Aucun tronçon de secours : 4 sites restent coupés

`reparer_pannes` généralise ce calcul à plusieurs pannes simultanées, avec le même coût qu'un Kruskal relancé sans les tronçons cassés.

7. Module 5 – Résilience et scénarios

Le module 5 (`src/indicateurs.py`) applique trois scénarios sur une copie du réseau, puis rassemble les indicateurs fournis par B (routes) et par C (fibre). Le retard moyen suit la définition de B : la moyenne sur toutes les demandes encore accessibles, retards nuls compris.

7.1 Tableau des trois scénarios

Jeu	Scénario	Demandes inaccessibles	Retard moyen	Retard max	Sites sensibles accessibles	Composantes de fibre	Coût de réparation
Petit	1. Situation normale	0 / 4	0	0	100 %	1	0
Petit	2. Fermeture de C1 → Q1	0 / 4	3,0 min	6,0 min	100 %	1	0
Petit	3. Panne de Q2 – Q3	0 / 4	0	0	100 %	2	9,0 k€
Réaliste	2. « Pont central »	0 / 25	0	0	100 %	1	0
Réaliste	3. Panne de TEL_01 – TEL_02	0 / 25	0	0	100 %	1	0
Généré	2. Fermeture de S0001 → S0002	0 / 50	0,3 min	4,6 min	100 %	1	0
Généré	3. Panne de S0001 – S0776	0 / 50	0	0	100 %	2	11,2 k€

Sur le petit jeu, les résultats correspondent aux calculs faits à la main. Sur le jeu réaliste, en revanche, les deux scénarios écrits au départ dans `scenarios.csv` ne montrent rien : la route du « pont central » n'est empruntée par aucun itinéraire, et le tronçon TEL_01 – TEL_02 n'est pas dans l'arbre couvrant. Choisir les scénarios à la main ne suffit donc pas à trouver les points faibles. Avec `--scenario`, fermer CAS_04 ↔ QUA_084 donne 4,7 min de retard moyen et 50,7 min au maximum ; la panne du tronçon de même nom donne 2 composantes et 40,0 k€ de réparation.

7.2 Test de résistance

Pour les trouver de façon systématique, le programme coupe **chaque** route utilisée par au moins un itinéraire, dans les deux sens, recalcule tous les trajets et additionne les minutes perdues. Couper une route que personne n'emprunte ne peut retarder personne : seules 53 routes du jeu réaliste sont donc testées. Un test vérifie que ce raccourci donne le même classement que couper toutes les routes. Le même principe s'applique à chacun des 119 tronçons de l'arbre de fibre. La commande `--operation points_faibles` régénère ces résultats.

Rang	Route coupée	Demandes retardées	Retard cumulé	Retard maximal
1	CAS_04 ↔ QUA_084	4	117,5 min	50,7 min
2	CAS_01 ↔ QUA_046	5	71,9 min	29,0 min
3	QUA_083 ↔ QUA_084	2	66,0 min	37,5 min

La route la plus empruntée (CAS_01 ↔ QUA_046, 5 itinéraires) n'est pas la plus critique : couper CAS_04 ↔ QUA_084 retarde moins de demandes, mais beaucoup plus longtemps, faute de détour court. Il faut donc recalculer, pas seulement compter les passages. Côté fibre, ECO_05 – QUA_074 et QUA_074 – QUA_084 n'ont **aucun** raccordement possible ; pour les autres tronçons, le pire surcoût est de 24,3 k€.

7.3 Les coupures se combinent

Quand plusieurs routes sont coupées en même temps, leurs effets ne s'additionnent pas. Avec les trois routes les plus critiques coupées, nous avons comparé les trois façons d'en rouvrir une seule :

Route rouverte en premier	Retard cumulé restant	Demandes inaccessibles
CAS_01 ↔ QUA_046	118,6 min	0
CAS_04 ↔ QUA_084	137,9 min	0
QUA_083 ↔ QUA_084	189,4 min	0

Contre l'intuition, il vaut mieux rouvrir d'abord CAS_01 ↔ QUA_046, et non la route la plus critique prise seule. Les deux autres coupures étant proches l'une de l'autre, leurs détours se recouvrent. Une décision de crise doit donc être évaluée en recalculant la situation complète.

8. Benchmarks et analyse critique

8.1 Protocole

Les mesures (`src/benchmarks.py`) suivent les règles du sujet.

- **Seul l'algorithme est chronométré** (`time.perf_counter()`), hors chargement, génération et affichage.
- **5 répétitions** (moyenne, médiane, écart-type) après 3 passes d'échauffement : sans elles, les premières passes étaient jusqu'à deux fois plus lentes.
- **Les mêmes graphes pour tous les algorithmes**, connexes, avec leur graine notée dans le CSV.
- **Tailles 100 à 5000**, en creux (m = 3n) et dense (m = 20n, soit moins de 1 % des arêtes possibles : un graphe complet de 5000 sommets serait trop lourd en Python). Floyd–Warshall à part, de 25 à 400 sommets.
- **Machine :** Apple M2, Python 3.14 ; les temps absolus en dépendent.

8.2 Résultats à n = 5000

Algorithme	Creux (m = 15 000)	Dense (m = 100 000)	Complexité du sujet
DFS	8,4 ms	79,6 ms	O(|V| + |E|)
BFS	9,6 ms	85,0 ms	O(|V| + |E|)
Prim	12,5 ms	105,4 ms	O(|E| log |V|)
Kruskal	15,0 ms	118,4 ms	O(|E| log |E|)

Algorithme	Creux (m = 15 000)	Dense (m = 100 000)	Complexité du sujet
Dijkstra	21,9 ms	174,9 ms	O((|V| + |E|) log |V|)
Bellman–Ford	31,0 ms	215,9 ms	O(|V| · |E|)

Floyd–Warshall prend 1,1 s pour seulement 400 sommets.

8.3 Comparaison avec la théorie

Pour comparer aux complexités, on divise le temps mesuré par le nombre d'opérations théorique. Une courbe presque plate confirme la théorie. Pour BFS, DFS, Dijkstra et Bellman–Ford, chaque arête non orientée est lue dans les deux sens, soit 2m arcs.

- **Kruskal, Prim et Dijkstra** sont presque plats : entre 47 et 86 ns par opération sur toute la plage. La théorie est confirmée, avec une constante propre à Python.
- **Floyd–Warshall** se situe entre 17 et 28 ns par opération. Quand n double, son temps est multiplié par un facteur compris entre 5 et 10, proche de $8 = 2^3$ sur les grandes tailles. Pour n = 1000, il faudrait 10^9 opérations et un million de cases en mémoire, d'où sa limite à quelques centaines de sommets.
- **Bellman–Ford** est des centaines de fois plus rapide que son pire cas : de 4,9 ns par opération théorique à n = 100, il tombe à 0,2 ns à n = 5000. L'arrêt anticipé l'explique : sur ces graphes, il termine en une dizaine de passes au lieu de n − 1. O(|V| · |E|) est un pire cas, rarement atteint en pratique.
- **BFS et DFS** devraient être plats, mais leur coût par opération double presque en dense (de 208 à 415 ns pour le BFS). La section suivante en explique la cause.

8.4 L'écart du BFS : la mémoire

Une expérience complémentaire (`--operation localite`) garde le même graphe et le même algorithme, et change seulement la disposition des données en mémoire : arêtes créées dans un ordre aléatoire (« originale »), copie profonde rangée sommet par sommet (« copie »), ou rangées dans l'ordre où le BFS les lira (« ordre du BFS »). La médiane est retenue, sur 3 graines.

À algorithme identique, la disposition mémoire change le temps d'un facteur 2 à 3 sur les gros graphes. L'écart apparaît au-delà de 1000 à 2000 sommets, quand le graphe dépasse une dizaine de mégaoctets, soit la taille du cache L2 du processeur (16 Mo). Le rôle exact du cache est probable mais n'a pas été mesuré directement. La notation O(|V| + |E|) compte les opérations, mais toutes n'ont pas le même coût.

Les graphiques ci-dessous montrent ces mesures.

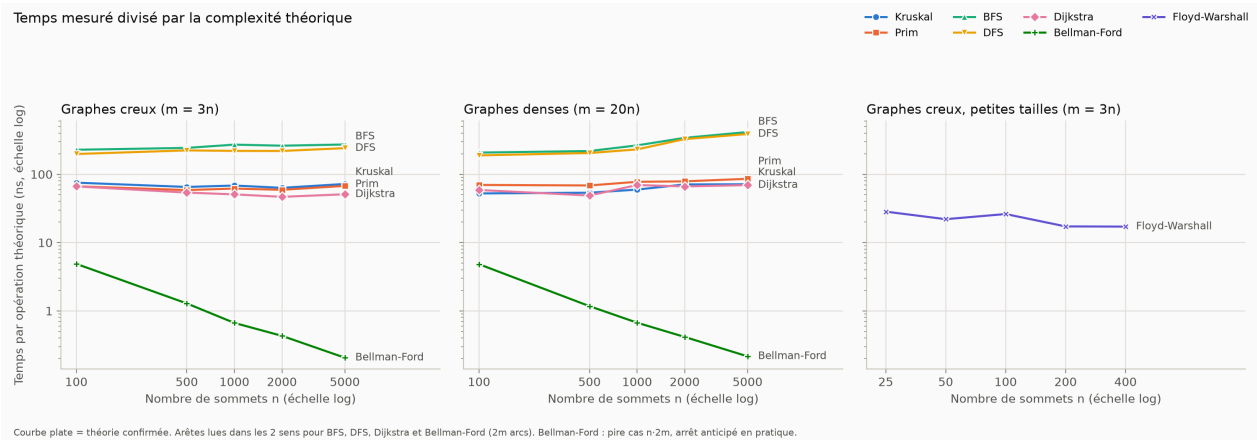


Figure 1 · Temps par opération théorique : une courbe plate confirme la complexité. Bellman–Ford descend, car il s'arrête bien avant son pire cas.

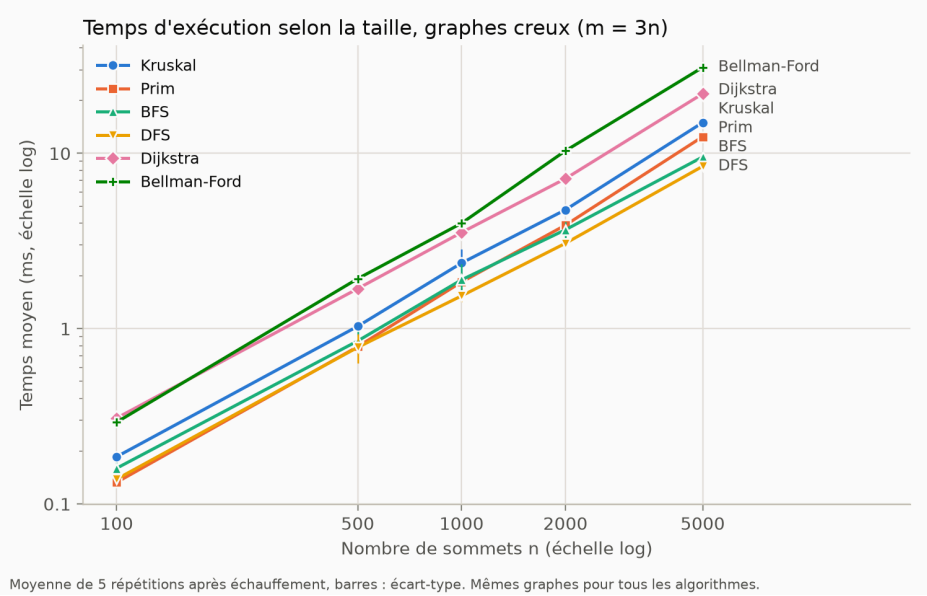


Figure 2 · Temps d'exécution selon la taille, graphes creux (m = 3n).

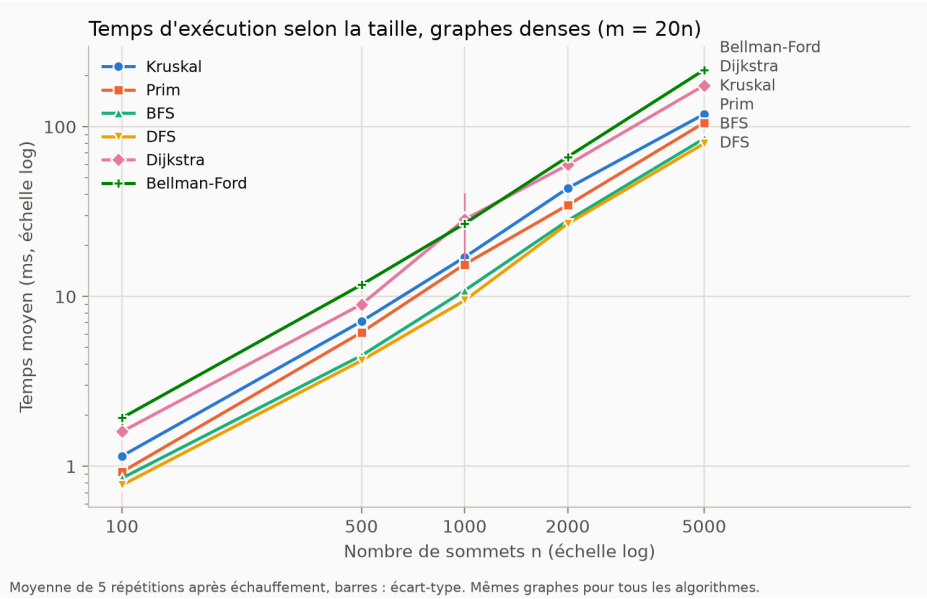


Figure 3 · Temps d'exécution selon la taille, graphes denses (m = 20n).

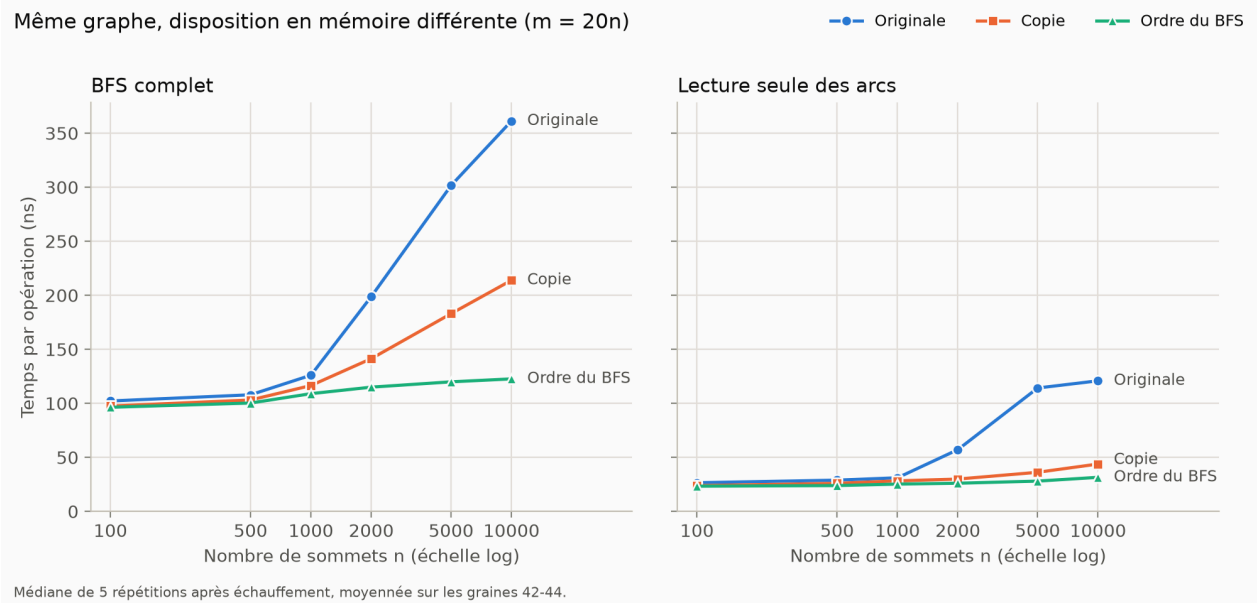


Figure 4 · Même graphe, disposition en mémoire différente (m = 20n).

9. L'outil : ligne de commande et site interactif

9.1 Ligne de commande

Comme le demande le sujet, l'interface permet de choisir un fichier, un scénario et une opération : `python3 -m src.cli --donnees <dossier> --scenario <nom> --operation <opération>`.

9.2 Le site

Le professeur ayant demandé une réalisation originale, nous avons ajouté un site local (`--operation site`). Il ne fait aucun calcul nouveau : le serveur (`src/site_web.py`, bibliothèque standard de Python) appelle les fonctions des trois parties et renvoie leurs résultats à la page.

- **La salle de crise** (`/salle`) affiche la ville comme une carte de nuit, où les routes sont des rangées de lampadaires. Un clic sur une route la bloque, un clic sur un tronçon de fibre simule une panne, et tous les indicateurs du module 5 sont recalculés. Une carte des fragilités colore chaque route selon le résultat du test de résistance.
- **La page Routes** (`/routes`) reprend la visualisation de la partie B : clic gauche pour le départ, clic droit pour l'arrivée, touches 1, 2 et 3 pour Dijkstra, Bellman–Ford et Floyd–Warshall (qui donnent la même durée), animation de l'exploration, durées des routes, flèches des sens uniques et modes accessibilité et connexité.
- **L'histoire** (`/`), « La nuit de l'orage », raconte une nuit de crise en neuf chapitres où l'utilisateur joue le régulateur. Elle commence par le diagnostic du module 1, puis enchaîne heure de pointe, orage, construction de la fibre par Kruskal et pannes. Deux décisions sont comparées à l'optimum calculé (section 7.3). La ville est dessinée en 3D isométrique, les véhicules roulent à la vitesse de leurs vrais temps de trajet, et une radio optionnelle utilise la synthèse vocale du navigateur.

L'histoire est inventée, mais aucun chiffre affiché n'est écrit à la main : tous sont calculés par le programme, et des tests vérifient que les bonnes réponses annoncées sont bien les optimums.

10. Tests et reproductibilité

377 tests automatiques passent avec `pytest`. Quand pygame n'est pas installé, les tests de la fenêtre de B sont sautés et il en reste 357.

Les tests couvrent les cas limites demandés : graphe vide, sommet isolé, source inexistante, toutes les routes fermées, égalités de coûts et graphe non connexe. Trois techniques renforcent leur valeur : des réponses indépendantes du code testé (calculs à la main, force brute), des graphes aléatoires à graine fixe, et la comparaison entre algorithmes.

Reproductibilité. Toutes les graines aléatoires sont fixées et notées, y compris dans le CSV des benchmarks. Les jeux réaliste et généré se reconstruisent à l'identique, ce que vérifient des tests. Les scénarios sont toujours appliqués sur une copie, ce que vérifient aussi des tests. Seuls les temps de mesure varient légèrement d'une exécution à l'autre (moins de 5 % au-delà de 1000 sommets) ; les conclusions restent les mêmes. Les commandes du README régénèrent tous les résultats de ce rapport.

11. Limites et recommandations

11.1 Limites du modèle

Les **données sont synthétiques** : les conclusions valent pour Novapolis, pas pour une ville réelle. Les **temps sont fixes** (l'heure de pointe est un second jeu de temps fixes) et ne suivent pas le trafic réel. La caserne retenue est la plus proche, **sans tenir compte des véhicules disponibles ni des priorités** des demandes. Une route en travaux reste praticable. Les coûts de fibre sont des estimations, et l'on ne raccorde qu'avec un tronçon déjà listé dans `fibre.csv`. Enfin, les temps mesurés dépendent de la machine : seules les comparaisons entre algorithmes sont valables, et Floyd–Warshall, en $O(n^3)$, reste limité à quelques centaines de sommets.

11.2 Recommandations pour la ville

1. **Prévoir un itinéraire de secours autour de CAS_04 ↔ QUA_084.** Sa fermeture coûte 117,5 min de retard cumulé et jusqu'à 50,7 min sur une seule demande.
2. **Doubler les tronçons de fibre ECO_05 – QUA_074 et QUA_074 – QUA_084.** Aucun raccordement n'est possible aujourd'hui en cas de panne : 4 sites seraient coupés.
3. **Remplacer les scénarios écrits à la main par le test de résistance.** Le « pont central » et la panne TEL_01 – TEL_02 ne révèlent rien, alors que le test trouve de vrais points faibles.
4. **Évaluer les décisions de crise sur la situation complète.** Quand plusieurs routes sont coupées, rouvrir la route la plus critique prise seule n'est pas toujours le meilleur choix (section 7.3).

12. Relevé des contributions

Le travail a été réparti en trois rôles, puis intégré sur une branche commune. Chacun a testé son propre module, et les sections 7, 10 et 11 sont communes. Chaque membre doit pouvoir expliquer tous les algorithmes.

Membre	Code	Sections du rapport
Jakub Potaczala, A · Fondations	`io_donnees.py`, `graphe.py`, `generateur.py`, `creer_jeu_realiste.py`, `cli.py`, `indicateurs.py` ; jeux petit, réaliste et généré ; README	1, 2

Membre	Code	Sections du rapport
Royston Gnanapragasam, B · Routes	`parcours.py`, `chemins.py`, `visu.py` (visualisation pygame) ; indicateurs routes du module 5	3, 4
Ravichandran Arumugadas, C · Fibre et mesures	`arbres.py`, `absorbants.py`, `benchmarks.py` ; jeux `egalites` et `absorbants` ; indicateurs fibre et correction du module 5 ; site (`site_web.py`, `salle_de_crise.py`, `recit.py`)	5, 6, 8, 9